



Universidad de Guadalajara
Centro Universitario de Ciencias Exactas e Ingenierías

ACTIVIDAD II - Compuertas Logicas

Documento elaborado por: Aaron Ricardo Segura Gutiérrez

Arquitectura de Computadoras
Profesor:
Jorge Ernesto Lopez Arce Delgado
SECCIÓN D05 2026A

Unidad Lógica - Implementación de compuertas lógicas

El presente trabajo se realiza con el fin de investigar, de forma general, el concepto de FPGA, compuertas lógicas y sus implementaciones, así como el modo de uso y sintaxis de forma digital a través de lenguajes como VHDL o Verilog.

Se investigará:

¿Qué es una FPGA?

Un FPGA o Field Programable Gate Array es un conjunto de circuitos integrados, como puede ser cualquier chip, que está pensado para entregarse al cliente sin configurarlo para que luego cada uno lo personalice y programe según la tarea que necesite que haga esa pieza, una vez esta ya se ha fabricado. Es como si fabricaran un procesador y luego lo configuraran en función de lo que se necesite, en vez de enfocarlo a una tarea o un uso específico durante el proceso de producción.

Los FPGAs suelen contener diferentes bloques de procesamiento en su interior, y diferentes conectores para poder configurar los diferentes bloques de lógica, pudiendo hacer operaciones complejas o pudiendo funcionar como simples puertas lógicas. También dependiendo del módulo en su interior podremos encontrar memoria, especialmente en los módulos de lógica más complejos y avanzados.

¿Cómo se relaciona con Verilog?

La implementación de la Lógica Digital se realiza mediante programación.

La programación de FPGAs se realiza en un lenguaje especialmente diseñado llamado Hardware Description Language.

Los lenguajes de programación convencionales como C o C++ cuyas versiones modificadas se pueden utilizar para programar los microcontroladores sólo pueden enviar una determinada instrucción al chip y no cambiar la lógica dentro del chip.

Es en esta parte donde tenemos a Verilog, estandarizado como IEEE 1364, es un lenguaje de descripción de hardware (HDL) utilizado para modelar sistemas electrónicos.

Verilog es un acrónimo de las palabras “verificación” y “lógica”.

Tenga en cuenta que VHDL no es una abreviatura de Verilog HDL: Verilog y VHDL son dos HDL diferentes. Sin embargo, tienen más similitudes que diferencias.

Verilog se puede utilizar para modelar y simular aspectos del sistema completo que contiene uno o más ASIC o FPGA. Esta puede ser una descripción completamente funcional del sistema que permita validar la especificación antes de comenzar el diseño detallado. Alternativamente, esta puede ser una descripción parcial que abstraiga ciertas propiedades del sistema, como un modelo de rendimiento para detectar cuellos de botella en el rendimiento del sistema.

Investigación sobre compuertas lógicas:

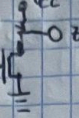
Aaron Ricardo Segura Gutiérrez

NOT $\rightarrow Z = \bar{A}$

A	Z
0	1
1	0

A $\rightarrow$ $\bar{A}$
 $Z = \bar{A}$

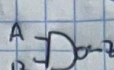
Transistores



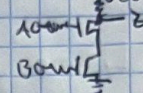
NAND $\rightarrow Z = \overline{A \cdot B}$

A	B	Z
0	0	1
0	1	1
1	0	1
1	1	0

Simbología



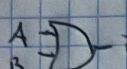
Transistores



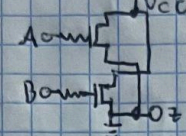
OR $\rightarrow Z = A + B$

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

Simbología



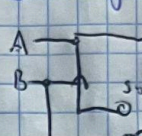
Transistores



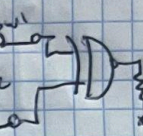
XNOR $\rightarrow Z = \overline{A \oplus B}$

A	B	Z
0	0	1
0	1	0
1	0	0
1	1	1

Simbología



Transistores



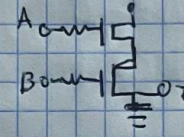
AND $\rightarrow Z = A \cdot B$

A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

Simbología



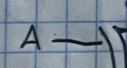
Transistores



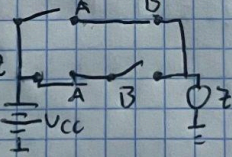
XOR $\rightarrow Z = A \oplus B$

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	0

Simbología



Transistores



NOR $\rightarrow Z = \overline{A + B}$

A	B	Z
0	0	1
0	1	0
1	0	0
1	1	0

Simbología



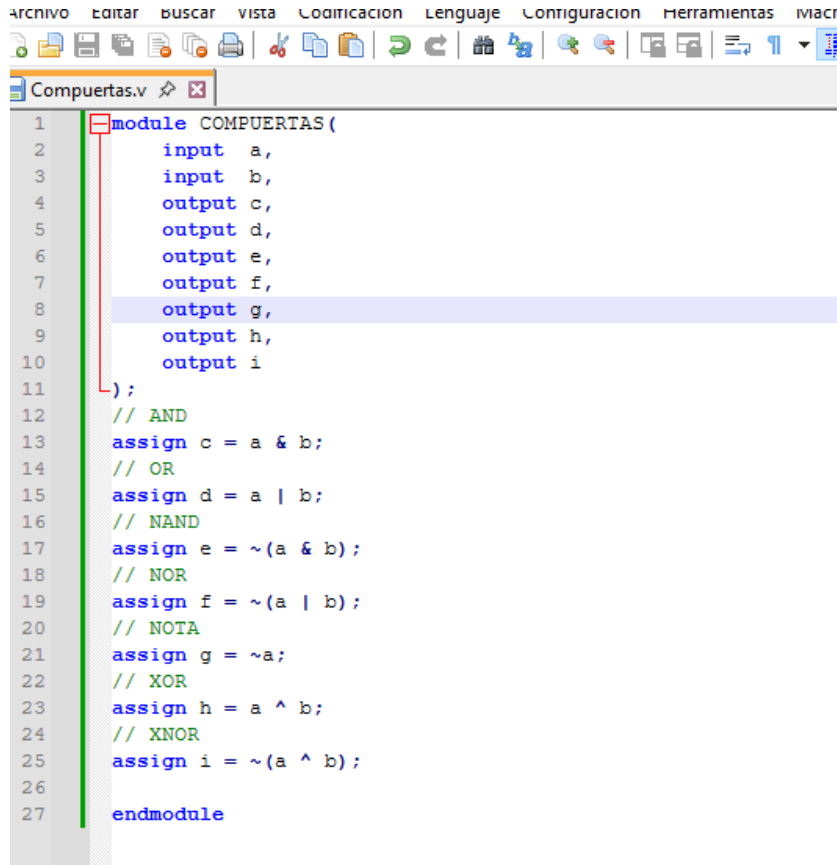
Transistores



ENTREGABLES

Se entregara un archivo con extensión “.v” de Verilog o “.vhd” en caso de código VHDL.

Resultado:



```
1 module COMPUERTAS(  
2     input a,  
3     input b,  
4     output c,  
5     output d,  
6     output e,  
7     output f,  
8     output g,  
9     output h,  
10    output i  
11 );  
12 // AND  
13 assign c = a & b;  
14 // OR  
15 assign d = a | b;  
16 // NAND  
17 assign e = ~(a & b);  
18 // NOR  
19 assign f = ~(a | b);  
20 // NOT  
21 assign g = ~a;  
22 // XOR  
23 assign h = a ^ b;  
24 // XNOR  
25 assign i = ~(a ^ b);  
26  
27 endmodule
```

